



Faculté des Sciences et Technologies (FST)

Rapport du travail de Laboratoire N° 3_ Modélisation Mathématique

Etudiant : Donsam Jean Gabard NOEL

Professeur : Ismaël SAINT AMOUR

Niveau : L3

18 Avril 2026

TD1 — Réseau social intelligent

Construire un graphe représentant des utilisateurs :

sommets = utilisateurs

arêtes = relations

Objectifs

1. créer un réseau de 20 utilisateurs
2. trouver les personnes les plus influentes
3. calculer le degré de chaque nœud
4. détecter les communautés

```
2 #--- TD1 - Réseau social intelligent ---
3
4 import networkx as nx
5 import matplotlib.pyplot as plt
6
7 # --- Créer un réseau de 20 utilisateurs ---
8 G = nx.Graph()
9
10 G.add_nodes_from([f"U{i}" for i in range(1, 21)])
11
12 G.add_edges_from([
13     ("U1", "U2"), ("U1", "U3"), ("U1", "U4"),
14     ("U2", "U5"), ("U2", "U6"), ("U2", "U7"),
15     ("U3", "U5"), ("U3", "U8"), ("U3", "U9"),
16     ("U4", "U9"), ("U4", "U10"),
17     ("U5", "U11"), ("U5", "U12"),
18     ("U6", "U12"), ("U6", "U13"),
19     ("U7", "U13"), ("U7", "U14"),
20     ("U8", "U15"), ("U8", "U16"),
21     ("U9", "U16"), ("U9", "U17"),
22     ("U10", "U17"), ("U10", "U18"),
23     ("U11", "U19"), ("U11", "U20"),
24     ("U12", "U19"), ("U13", "U20"),
25     ("U14", "U18"), ("U15", "U20"),
26     ("U16", "U19"), ("U18", "U20"),
27 ])
```

```
28
29 # --- Calculer le degré de chaque nœud ---
30 print("Degré de chaque utilisateur :")
31 for node, deg in sorted(G.degree(), key=lambda x: x[1], reverse=True):
32     print(f" {node} : {deg}")
33
34 # --- Trouver les personnes les plus influentes ---
35 betweenness = nx.betweenness_centrality(G)
36 print(f"\nTop 5 utilisateurs les plus influents (betweenness) :")
37 top5 = sorted(betweenness, key=betweenness.get, reverse=True)[:5]
38 for u in top5:
39     print(f" {u} : {betweenness[u]:.3f}")
40
41 # --- Détecter les communautés ---
42 communities = list(nx.community.greedy_modularity_communities(G))
43 print(f"\n{len(communities)} communautés détectées :")
44 for i, c in enumerate(communities, 1):
45     print(f" Communauté {i} : {sorted(c)}")
46
47 # --- Visualisation ---
48 color_map = {}
49 colors = ["#FF6868", "#4ECDC4", "#4587D1"]
50 for i, comm in enumerate(communities):
51     for node in comm:
52         color_map[node] = colors[i]
53
```

```

54 node_colors = [color_map[n] for n in G.nodes()]
55 node_sizes = [300 + G.degree(n) * 100 for n in G.nodes()]
56 pos = nx.spring_layout(G, seed=42)
57
58 nx.draw(G, pos,
59         with_labels=True,
60         node_color=node_colors,
61         node_size=node_sizes,
62         font_size=7,
63         edge_color="aaa")
64
65 plt.title("TD1 - Réseau social (taille  $\propto$  degré, couleur = communauté)")
66 plt.savefig("TD1_reseau_social.png", dpi=150, bbox_inches='tight')
67 plt.show()
68

```

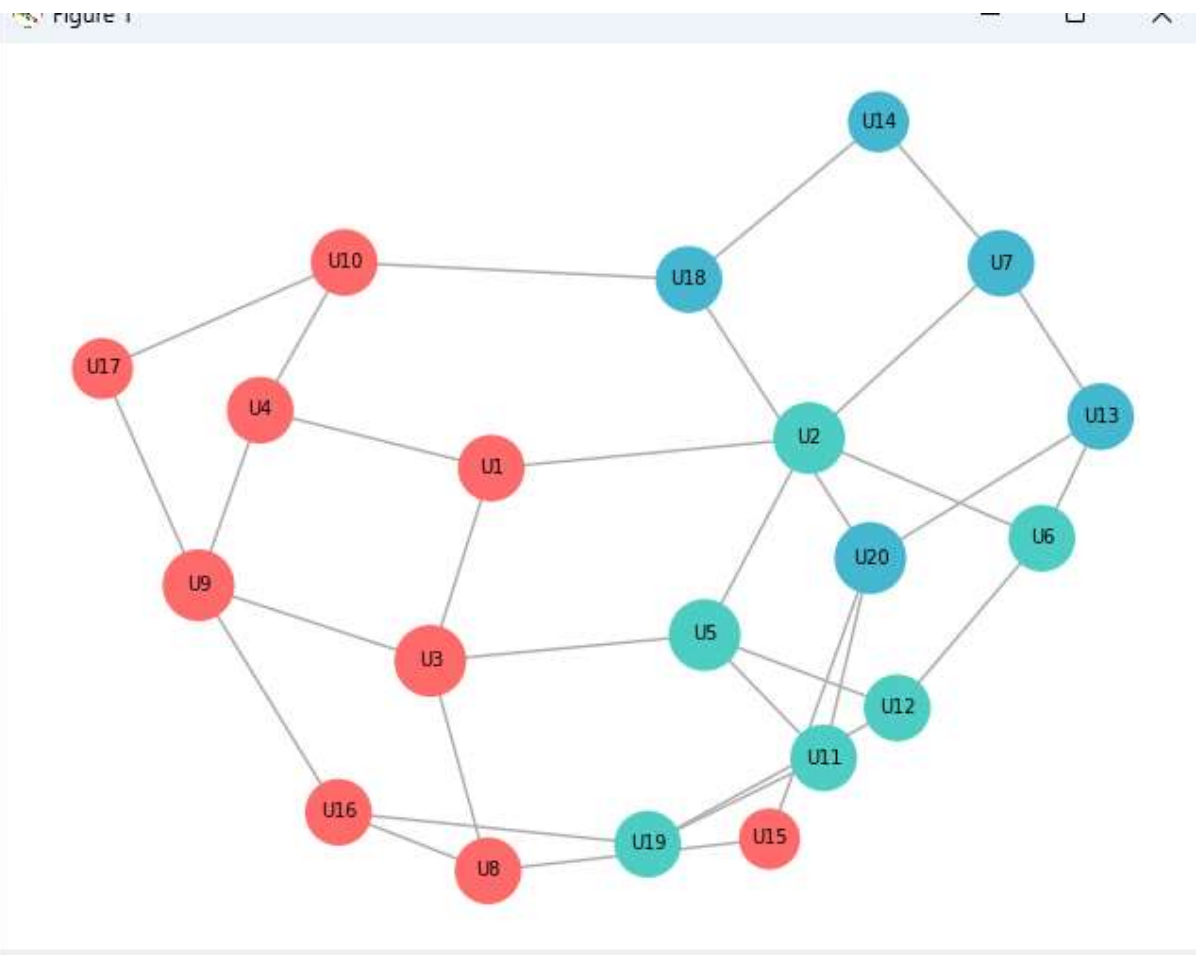
```

Degré de chaque utilisateur :
U2 : 4
U3 : 4
U5 : 4
U9 : 4
U20 : 4
U1 : 3
U4 : 3
U6 : 3
U7 : 3
U8 : 3
U10 : 3
U11 : 3
U12 : 3
U13 : 3
U16 : 3
U18 : 3
U19 : 3
U14 : 2
U15 : 2
U17 : 2

Top 5 utilisateurs les plus influents (betweenness) :
U20 : 0.192
U3 : 0.150
U2 : 0.148
U5 : 0.137
U10 : 0.133

3 communautés détectées :
Communauté 1 : ['U1', 'U10', 'U15', 'U16', 'U17', 'U3', 'U4', 'U8', 'U9']
Communauté 2 : ['U11', 'U12', 'U19', 'U2', 'U5', 'U6']
Communauté 3 : ['U13', 'U14', 'U18', 'U20', 'U7']

```



TD2 — Chaîne de Markov (météo + économie)

Modéliser deux systèmes :

1. météo

Soleil

Pluie

Nuage

2. économie

Hausse

Stable

Baisse

Objectifs

1. construire la matrice de transition

2. simuler 50 à 100 jours

3. calculer la distribution stationnaire

4. comparer simulation vs théorie

```
2 # --- TD2 - Chaîne de Markov : Météo + Économie ---
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 # --- Construire les matrices de transition ---
8
9 # Météo : états = "Soleil, Pluie, Nuage"
10 etats_meteo = ["Soleil", "Pluie", "Nuage"]
11 M_meteo = np.array([[0.6, 0.2, 0.2],
12                    [0.3, 0.4, 0.3],
13                    [0.4, 0.3, 0.3]])
14
15 # Économie : états = "Hausse, Stable, Baisse"
16 etats_eco = ["Hausse", "Stable", "Baisse"]
17 M_eco = np.array([[0.5, 0.3, 0.2],
18                  [0.2, 0.5, 0.3],
19                  [0.1, 0.4, 0.5]])
20
21 # --- Simuler 100 jours ---
22 jours = 100
23
24 # État initial : Soleil certain / Stable certain
25 X_meteo = np.array([1.0, 0.0, 0.0])
26 X_eco = np.array([0.0, 1.0, 0.0])
27
```

```

28 historique_meteo = [X_meteo.copy()]
29 historique_eco = [X_eco.copy()]
30
31 for i in range(jours):
32     X_meteo = X_meteo @ M_meteo
33     X_eco = X_eco @ M_eco
34     historique_meteo.append(X_meteo.copy())
35     historique_eco.append(X_eco.copy())
36
37 historique_meteo = np.array(historique_meteo)
38 historique_eco = np.array(historique_eco)
39
40 print("\nAprès 100 jours - Météo :")
41 for i, e in enumerate(etats_meteo):
42     print(f" {e} : {X_meteo[i]*100:.1f}%")
43
44 print("\nAprès 100 jours - Économie :")
45 for i, e in enumerate(etats_eco):
46     print(f" {e} : {X_eco[i]*100:.1f}%")
47
48 # --- Calculer la distribution stationnaire ---
49 def distribution_stationnaire(M):
50     n = M.shape[0]
51     A = np.vstack([M.T - np.eye(n), np.ones(n)])
52     b = np.zeros(n + 1); b[-1] = 1
53     pi, _, _, _ = np.linalg.lstsq(A, b, rcond=None)
54     return pi

```

```

56 pi_meteo = distribution_stationnaire(M_meteo)
57 pi_eco = distribution_stationnaire(M_eco)
58
59 print("\nDistribution stationnaire - Météo :")
60 for i, e in enumerate(etats_meteo):
61     print(f" {e} : {pi_meteo[i]*100:.1f}%")
62
63 print("\nDistribution stationnaire - Économie :")
64 for i, e in enumerate(etats_eco):
65     print(f" {e} : {pi_eco[i]*100:.1f}%")
66
67 # --- Comparer simulation vs théorie ---
68 fig, axes = plt.subplots(2, 2, figsize=(14, 9))
69 fig.suptitle("TD2 - Chaînes de Markov : Météo & Économie", fontsize=14, fontweight='bold')
70
71 colors_meteo = [{"e": "#FFD700", "c": "#4169E1", "t": "#A9A9A9"}]
72 colors_eco = [{"e": "#2ECC71", "c": "#3498DB", "t": "#E74C3C"}]
73
74 # Courbes de convergence - Météo
75 ax = axes[0][0]
76 for i, (e, c) in enumerate(zip(etats_meteo, colors_meteo)):
77     ax.plot(historique_meteo[:, i], color=c, linewidth=2, label=e)
78     ax.axhline(pi_meteo[i], color=c, linestyle='--', alpha=0.5)
79 ax.set_title("Convergence Météo\n(traits = simulation | tirets = théorie)")
80 ax.set_xlabel("Jours"); ax.set_ylabel("Probabilité")
81 ax.legend(); ax.grid(alpha=0.3)

```

```

83 # Courbes de convergence - Économie
84 ax = axes[0][1]
85 for i, (e, c) in enumerate(zip(etats_eco, colors_eco)):
86     ax.plot(historique_eco[:, i], color=c, linewidth=2, label=e)
87     ax.axhline(pi_eco[i], color=c, linestyle='--', alpha=0.5)
88 ax.set_title("Convergence Économie\n(traits = simulation | tirets = théorie)")
89 ax.set_xlabel("Jours"); ax.set_ylabel("Probabilité")
90 ax.legend(); ax.grid(alpha=0.3)
91
92 # Barres simulation vs théorie - Météo
93 ax = axes[1][0]
94 x, w = np.arange(3), 0.35
95 ax.bar(x - w/2, historique_meteo[-1] * 100, w, label="Simulation", color=colors_meteo, edgecolor='black', alpha=0.85)
96 ax.bar(x + w/2, pi_meteo * 100, w, label="Théorie", color=colors_meteo, edgecolor='black', alpha=0.4, hatch='//')
97 ax.set_xticks(x); ax.set_xticklabels(etats_meteo)
98 ax.set_title("Météo : Simulation vs Théorie"); ax.set_ylabel("%")
99 ax.legend(); ax.grid(axis='y', alpha=0.3)
100
101 # Barres simulation vs théorie - Économie
102 ax = axes[1][1]
103 ax.bar(x - w/2, historique_eco[-1] * 100, w, label="Simulation", color=colors_eco, edgecolor='black', alpha=0.85)
104 ax.bar(x + w/2, pi_eco * 100, w, label="Théorie", color=colors_eco, edgecolor='black', alpha=0.4, hatch='//')
105 ax.set_xticks(x); ax.set_xticklabels(etats_eco)
106 ax.set_title("Économie : Simulation vs Théorie"); ax.set_ylabel("%")
107 ax.legend(); ax.grid(axis='y', alpha=0.3)
108
109 plt.tight_layout()
110 plt.show()

```

Après 100 jours – Météo :

Soleil : 46.5%

Pluie : 28.2%

Nuage : 25.4%

Après 100 jours – Économie :

Hausse : 23.6%

Stable : 41.8%

Baisse : 34.5%

Distribution stationnaire – Météo :

Soleil : 46.5%

Pluie : 28.2%

Nuage : 25.4%

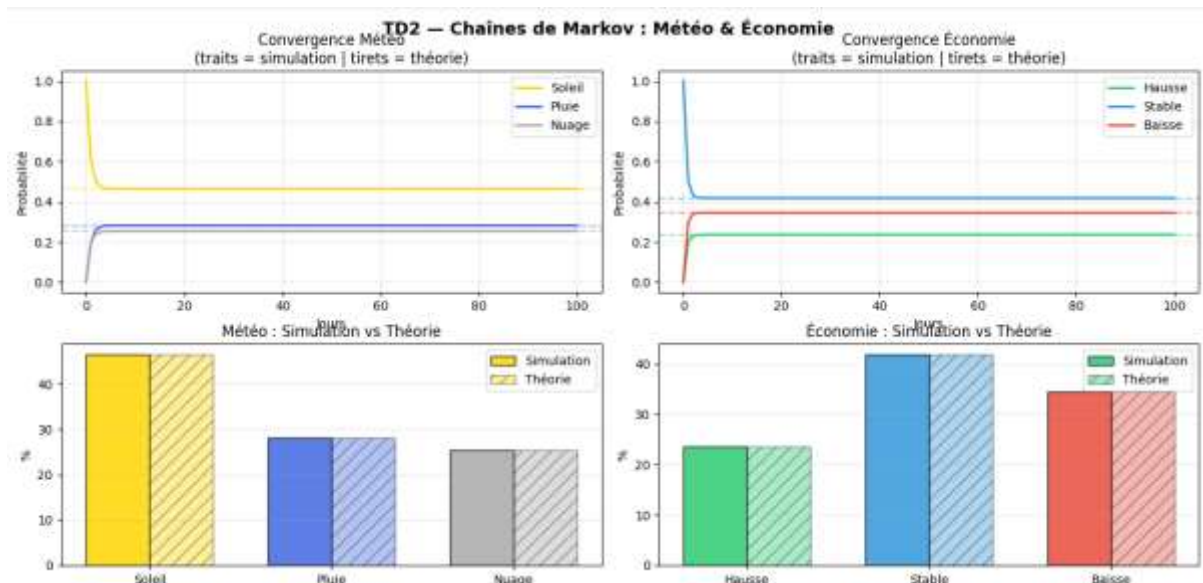
Distribution stationnaire – Économie :

Hausse : 23.6%

Stable : 41.8%

Baisse : 34.5%

PS C:\Users\Christy>



TD3 — Réseau de transport (type métro)

Créer un réseau de métro simplifié.

Objectifs

1. créer 15 à 25 stations
2. relier les stations
3. associer des temps de trajet
4. calculer itinéraires optimaux
5. afficher les lignes

```
2 # TD3 - Réseau de transport (type métro)
3
4 import networkx as nx
5 import matplotlib.pyplot as plt
6
7 # --- Créer 21 stations ---
8 G = nx.Graph()
9
10 # --- Relier les stations et associer des temps de trajet ---
11
12 G.add_weighted_edges_from([
13     # Ligne 1 (Rouge) : Anse-a-Pitre → Saint-Marc
14     ("Anse-a-Pitre", "Belle-Anse", 5),
15     ("Belle-Anse", "Marigot", 4),
16     ("Marigot", "Cayes-Jacmel", 3),
17     ("Cayes-Jacmel", "Jacmel", 4),
18     ("Jacmel", "Lavalle", 5),
19     ("Lavalle", "Leogane", 3),
20     ("Leogane", "Port-au-Prince", 4),
21     ("Port-au-Prince", "Saint-Marc", 6),
22
23     # Ligne 2 (Bleue) : Mirebalais → Port-Salut
24     ("Mirebalais", "Hinche", 6),
25     ("Hinche", "Lamontagne", 5),
26     ("Lamontagne", "Port-au-Prince", 7),
27     ("Port-au-Prince", "Carrefour", 3),
28     ("Carrefour", "Jeremi", 8),
29     ("Jeremi", "Port-Salut", 5),
30
31     # Ligne 3 (Verte) : Petion-Ville → Port-de-Paix
32     ("Petion-Ville", "Monchil", 4),
33     ("Monchil", "Post-Marchand", 5),
34     ("Post-Marchand", "Gonaïves", 6),
35     ("Gonaïves", "Cap-Haïtien", 4),
36     ("Cap-Haïtien", "Port-de-Paix", 5),
37
38     # Connexions supplémentaires
39     ("Port-au-Prince", "Petion-Ville", 2),
40     ("Saint-Marc", "Gonaïves", 5),
41 ])
42
```



```

43 # --- Calculer les itinéraires optimaux ---
44 trajets = [
45     ("Anse-a-Pitre", "Cap-Haitien"),
46     ("Jacmel", "Port-de-Paix"),
47     ("Mirebalais", "Port-Salut"),
48     ("Hinche", "Gonaïves"),
49     ("Leogane", "Petion-Ville"),
50 ]
51
52 print("Itinéraires optimaux :")
53 for depart, arrivee in trajets:
54     path = nx.shortest_path(G, source=depart, target=arrivee, weight="weight")
55     distance = nx.shortest_path_length(G, source=depart, target=arrivee, weight="weight")
56     print(f"\n {depart} → {arrivee}")
57     print(f" Chemin : {' '.join(path)}")
58     print(f" Durée : {distance} min")
59
60 # --- Afficher les lignes ---
61 pos = nx.spring_layout(G, seed=42)
62
63 ligne1 = ["Anse-a-Pitre", "Belle-Anse", "Marigot", "Cayes-Jacmel",
64           "Jacmel", "Lavalie", "Leogane", "Port-au-Prince", "Saint-Marc"]
65 ligne2 = ["Mirebalais", "Hinche", "Lamontagne", "Port-au-Prince",
66           "Carrefour", "Jeremi", "Port-Salut"]
67 ligne3 = ["Petion-Ville", "Monchil", "Post-Marchand",
68           "Gonaïves", "Cap-Haitien", "Port-de-Paix"]
69

```

```

70
71 node_colors = []
72 for n in G.nodes():
73     if G.degree(n) >= 3:
74         node_colors.append("orange") # correspondance
75     elif n in ligne1:
76         node_colors.append("#FF6B6B") # rouge
77     elif n in ligne2:
78         node_colors.append("#6B9FFF") # bleu
79     else:
80         node_colors.append("#6BFF8A") # vert
81
82 nx.draw(G, pos,
83         with_labels=True,
84         node_color=node_colors,
85         node_size=900,
86         font_size=6,
87         edge_color="aaa")
88
89 labels = nx.get_edge_attributes(G, "weight")
90 nx.draw_networkx_edge_labels(G, pos, edge_labels=labels, font_size=6)
91
92 plt.title("Réseau de transport (orange = correspondance)")
93 plt.savefig("Réseau de transport.png", dpi=150, bbox_inches='tight')
94 plt.show()
95

```

Itinéraires optimaux :

```

Anse-a-Pitre → Cap-Haitien
Chemin : Anse-a-Pitre → Belle-Anse → Marigot → Cayes-Jacmel → Jacmel → Lavalie → Leogane → Port-au-Prince → Saint-Marc → Gonaïves → Cap-Haitien
Durée : 43 min

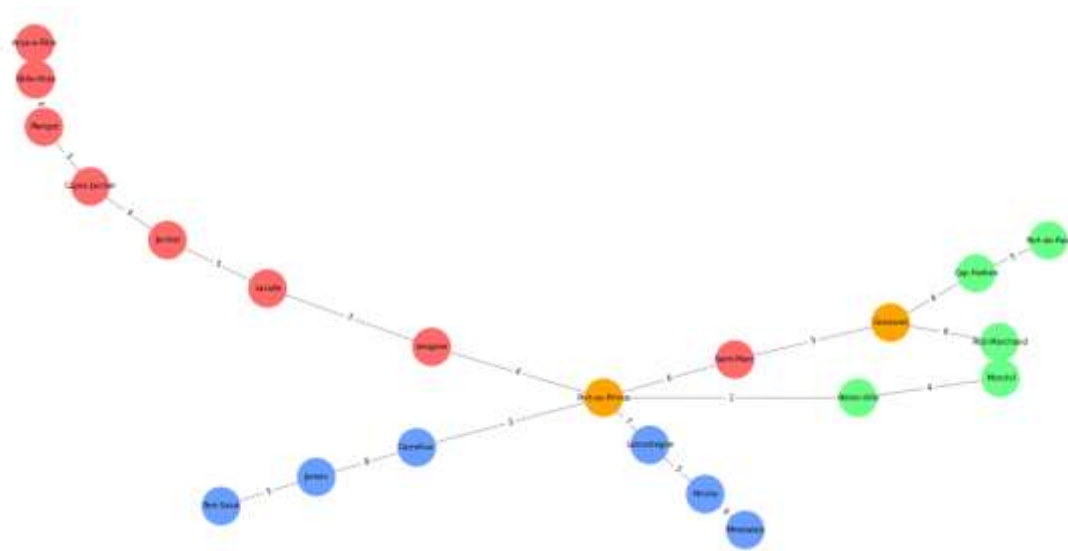
Jacmel → Port-de-Paix
Chemin : Jacmel → Lavalie → Leogane → Port-au-Prince → Saint-Marc → Gonaïves → Cap-Haitien → Port-de-Paix
Durée : 32 min

Mirebalais → Port-Salut
Chemin : Mirebalais → Hinche → Lamontagne → Port-au-Prince → Carrefour → Jeremi → Port-Salut
Durée : 34 min

Hinche → Gonaïves
Chemin : Hinche → Lamontagne → Port-au-Prince → Saint-Marc → Gonaïves
Durée : 23 min

Leogane → Petion-Ville
Chemin : Leogane → Port-au-Prince → Petion-Ville
Durée : 6 min

```



Conclusion

Ces TD ont permis d'appliquer les matrices et la théorie des graphes à des cas concrets : modélisation de réseaux sociaux (analyse des relations), simulation de systèmes probabilistes via les chaînes de Markov (météo, économie), et optimisation de trajets dans un réseau de transport. Chaque exercice a renforcé l'utilisation d'outils comme NumPy et NetworkX pour résoudre des problèmes réels, illustrant la polyvalence de ces concepts en informatique, économie et logistique.